1. $\forall c \in [a,b]$,

$$f(x) = f(c) + f'(c)(x-c) + \frac{f''(\xi)}{2}(x-c)^2,$$

$\exists \xi \in [c,x]$ if $c \le x$ or $\xi \in [x,c]$ if $c \ge x$

$f(a) = f(b) = 0$

$\Rightarrow \exists c_0 \in [a,b]$, s.t. $f'(c_0) = 0$

$$f(x) = f(c_0) + \frac{f''(\xi)}{2}(x-c_0)^2$$

$$\Rightarrow f(x) \le f(c_0) + \frac{1}{2}(x-c_0)^2 \max_{a \le x \le b} |f''(x)|$$

$$f(x) \le f(c_0) + \frac{1}{8}(b-a)^2 \max_{a \le x \le b} |f''(x)|$$

$f'(c_0) = 0 \Rightarrow$ If $\max_{a \le x \le b} |f(x)| = |f(x_0)|$,

Then x_0 is fit for c_0

$$\max_{a \le x \le b} |f(x)| \le f(x) - f(c_0) \le \frac{1}{8}(b-a)^2 \max_{a \le x \le b} |f''(x)|$$

2、

a、$P_2(x)=\frac{6}{0!}+\frac{0}{1!}x+\frac{0}{2!}x^2=6$

b、$R_2(x)=\frac{f'''(c)}{3!}x^3$

$R_2(0.5)=\frac{6}{3!}\times(0.5)^3=\frac{1}{8}$

c、$P_2(x)=1+3(x-1)+3(x-1)^2$

d、$R_2(0.5)=\frac{6}{6}(0.5-1)^3=-\frac{1}{8}$

3、

a、 $f'(x) = e^x + 1 > 0$

$f(1) = e - 3 < 0$

$f(2) = e^2 - 2 > 0$

By IVT, $f(x)$ has a unique root x_0 where $f(x_0) = 0$ and $x_0 \in (1, 2)$

b、 $n \geq \frac{\log(2-1) - \log(10^{-6})}{\log(2)} = 6 + \frac{\log 5}{\log 2}$

n is at least 7

Question 4

1.

Interval	Expected Root
[-0.3, -0.1]	Close to -0.2
[0.2, 0.4]	Close to 0.3
[1.1, 1.3]	Close to 1.2

2.

Interval	Expected Root
[-1, 2]	One of the roots in the question 1 above
[-2, 2]	One of the roots in the question 1 above

3.

The bisection method divides the intervals by haves so if the root is in the interval neatest to 0.3, then it won't be found out.

4. The program stops if the both conditions as follow are met.

Condition 1: The absolute value of the function at the midpoint < e^-6, which is small enough.

Condition 2: The times the iterator runs reach 10000, which is too big.

Code:

```
def bisection(function, a, b, tole=1e-6, max_it=10000):
    if function(a) * function(b) >= 0:
        print("No root in [", a, ",", b, "]")
        return None  # Root is not bracketed

    for time in range(max_it):
        midpt = (a + b) / 2
        midpt_valaue = function(midpt)
        if abs(midpt_valaue) < tole:
            return midpt
        elif function(a) * midpt_valaue < 0:
            b = midpt
        else:
            a = midpt
    return (a + b) / 2

def f(x):
    return (x + 0.2) * (x - 0.3) * (x - 1.2)

intervals = [(-0.3, -0.1), (0.2, 0.4), (1.1, 1.3)]
roots = [bisection(f, interval[0], interval[1]) for interval in intervals]
print("Roots: ", roots)

intervals_extended = [(-1, 2), (-2, 2)]
roots_extended = [bisection(f, interval[0], interval[1]) for interval in intervals_extended]
print("Roots:", roots_extended)
```

Question 5

Illustration:

The equation $f(x) = x^4 + 2x^2 - x - 3$ has a unique root in [1, 2]. To find this root using the fixed-point iteration method, we can transform the equation into the form x=g(x) using several algebraic manipulations. Each transformation leads to a different function g, and the effectiveness of each in converging to the root can vary. The

following functions are examples of such transformations:

$$g_1(x) = (3 + x - 2x^2)^{1/4}$$

$$g_2(x) = \left(\frac{x + 3 - x^4}{2}\right)^{1/2}$$

$$g_3(x) = \left(\frac{x + 3}{x^2 + 2}\right)^{1/2}$$

$$g_4(x) = \frac{3x^4 + 2x^2 + 3}{4x^3 + 4x - 1}$$

$$g_{\text{construct}}(x) = \frac{5 + x}{23 + x^2}$$

The original equation is rearranged in each case to form a fixed-point iteration function. The effectiveness of each function g(x) in converging to the root depends on its specific form and the properties of the function.

Starting with an initial guess of p_0 = 1, we apply fixed-point iteration to each of these transformations. The results of these iterations, particularly the convergence rates and the accuracy of the approximations, can be compared to evaluate the effectiveness of each method.

Table:

n	g_1	g_2	g_3	g_4	g_construct
0	1	1	1	1	1
1	1.189207115002721	1.224744871391589	1.154700538379251	1.142857142857128	0.25
2	1.080057752667562	0.993666159077481	1.116427409872122	1.124481690017893	0.227642276422764
3	1.149671430589382	1.228568645274987	1.126052233002257	1.124123163940188	0.226777844364604
4	1.107820529510259	0.987506429150886	1.123638884713254	1.124123029704334	0.226744208693955
5	1.133932284504730	1.232183418318806	1.124244496886007	1.124123029704315	0.226742899585614
6	1.118003117715782	0.981585828612739	1.124092552775951	1.124123029704315	0.226742848634320
7	1.127857163488397	1.235563208535450	1.124130676442744	1.124123029704315	0.226742846651264
8	1.121813166001129	0.975962460004536	1.124121111110316	1.124123029704315	0.226742846574082
9	1.125539874244780	1.238689029183278	1.124123511085903	1.124123029704315	0.226742846571078
10	1.123249432277918	0.970684690957617	1.124122908924049	1.124123029704315	0.226742846570961
15	1.124200050956794	1.246463945613536	1.124123029824411	1.124123029704315	0.226742846570956
20	1.124116233019905	0.950317401707698	1.124123029704196	1.124123029704315	0.226742846570956
25	1.124123629425607	1.254520930718409	1.124123029704315	1.124123029704315	0.226742846570956
30	1.124122976786142	0.939267690312917	1.124123029704315	1.124123029704315	0.226742846570956

Code:

```python
def fix_pt_method(func_g, pt, max_it = 1, it_num_list = [0]):
    results_list = []
    pt_use = pt
    for it_num in range(max_it):
        pt_new = func_g(pt_use)
        error = abs(pt_new - pt_use)

        if it_num in it_num_list:
            results_list.append((it_num, pt_use, pt_new, error))

        pt_use = pt_new

    return results_list

def g1(x):
    return (3 + x - 2*x**2)**(1/4)
def g2(x):
    return ((x + 3 - x**4) / 2)**(1/2)
def g3(x):
    return ((x + 3) / (x**2 + 2))**(1/2)
def g4(x):
    return (3*x**4 + 2*x**2 + 3) / (4*x**3 + 4*x - 1)
def g_construct(x):
    return (5 + x) / (21 + x**2)

init_p0 = 1
n_list = list(range(10)) + list(range(10, 31, 5))

results_g1 = fix_pt_method(g1, init_p0, 31, n_list)
results_g2 = fix_pt_method(g2, init_p0, 31, n_list)
results_g3 = fix_pt_method(g3, init_p0, 31, n_list)
results_g4 = fix_pt_method(g4, init_p0, 31, n_list)
results_g_construct = fix_pt_method(g_construct, init_p0, 31, n_list)

print("g1 results:", results_g1)
print("g2 results:", results_g2)
print("g3 results:", results_g3)
print("g4 results:", results_g4)
print("g_construct results:", results_g_construct)
```